2026-03-11

Center of Innovation for Sustainable Quantum AI

量子計算とテンソルネットワーク

#4: 大規模シミュレーション戦略

https://github.com/todo-group/tnqc-springschool

量子計算とテンソルネットワーク

- #4 大規模シミュレーション戦略
  - 量子コンピュータの古典シミュレーション
  - TN シミュレータ
  - Clifford シミュレータ

量子コンピュータの古典シミュレーション

なぜ量子コンピュータを古典的にシミュレートするのか？

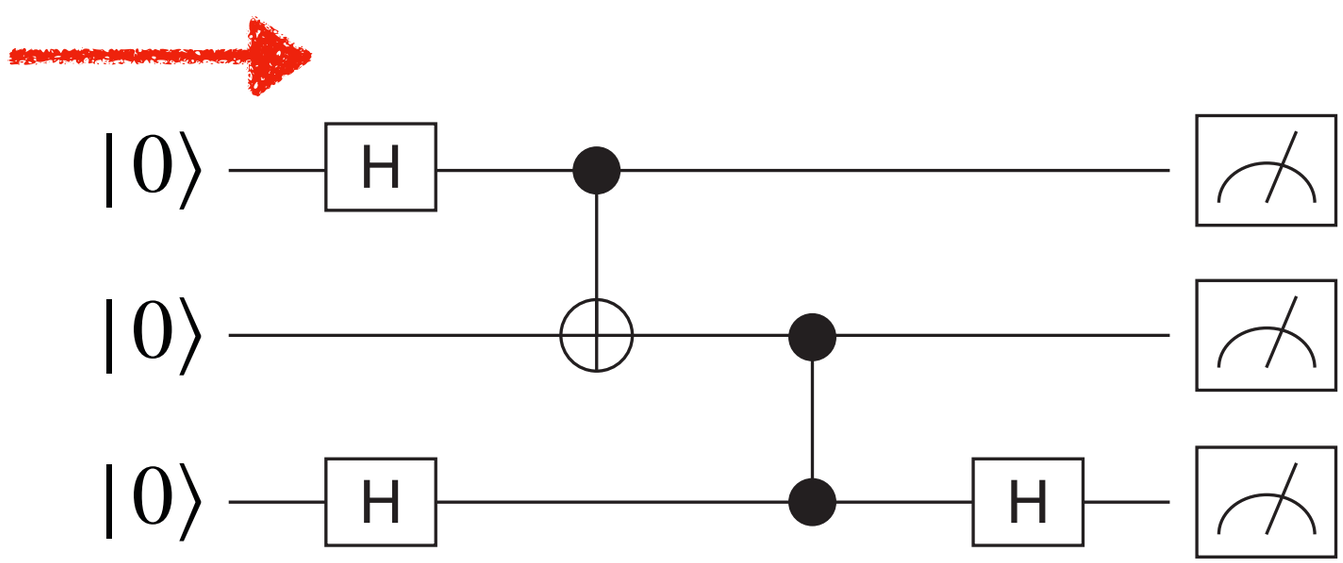
- 量子アルゴリズムの精度・性能評価
  - 量子古典ハイブリッドアルゴリズム, 量子機械学習, 量子誤り訂正
- 古典計算の限界を理解する
  - 量子超越性の本当の境界はどこか？
- 量子回路は典型的な量子多体系である
  - 量子回路のシミュレーション技法 → 物理学や他分野への応用
    - 超伝導, 量子液体, 素粒子, 原子核
    - 複雑系, 時系列データ, 機械学習
- 量子計算の本質を理解する
  - 別の形に書き換えることでのみ理解できる
- 古典計算機上で量子技術を活用する
  - 量子コンピュータが広く使われる前に, 誰でもどこでも量子計算を利用可能にする

量子回路の古典シミュレーション

- 量子回路は量子多体系の 1 つである
  - 量子多体系を古典計算機でシミュレートできるか？
    - → できる!
  - 量子系の古典シミュレーションの伝統的手法
    - Schrödinger シミュレーション
    - Feynman シミュレーション
    - Feynman サンプリング
  - 古典シミュレーションで何を求めたいのか？
    - 波動関数そのもの
    - 測定確率
    - 測定の期待値
    - 測定結果のサンプル、など

Schrödinger シミュレーション

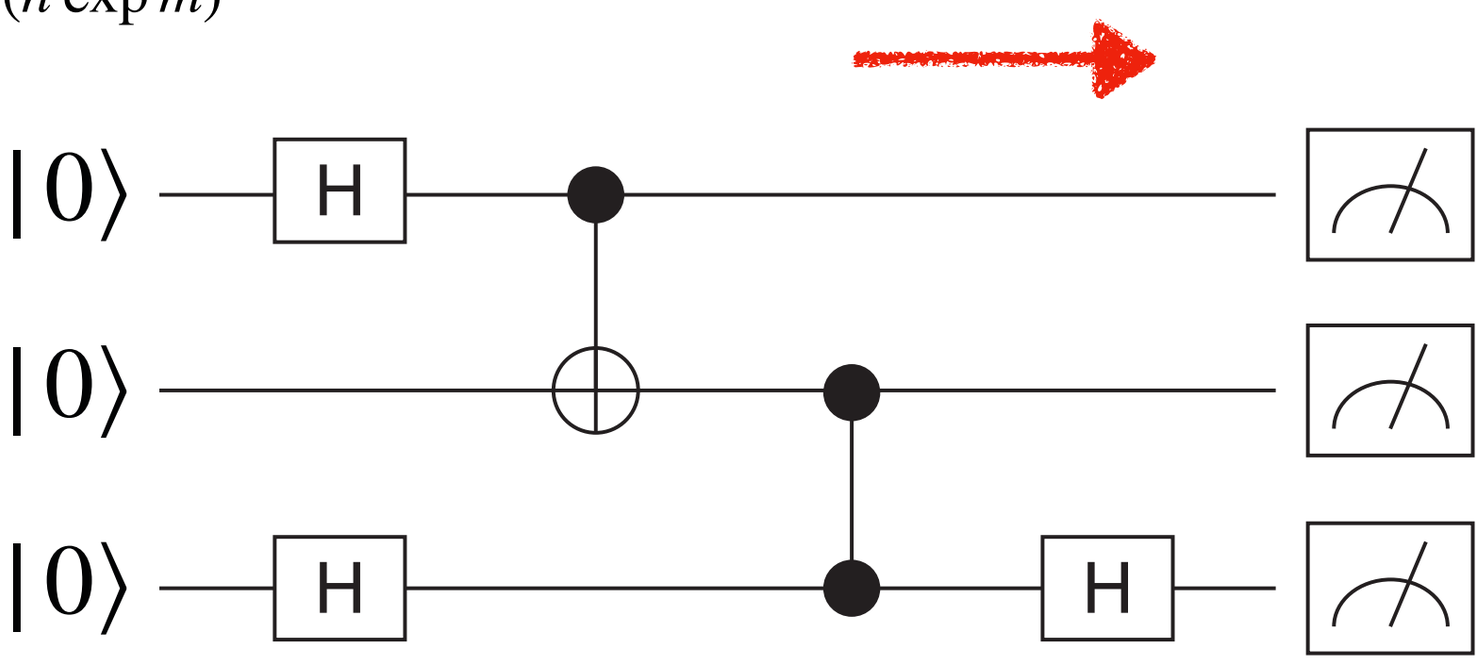
- 波動関数の係数（状態の重ね合わせ係数）をベクトルとして保持する
  - n 量子ビット → サイズ 2^n の複素ベクトル
    - 量子ゲート → $2^n \times 2^n$ ユニタリ行列（ただし通常は非常に疎）
    - ゲート操作 → 行列ベクトル積
- シミュレーションコスト
  - n 量子ビット, m ゲート
    - メモリコスト: $O(\exp n)$　　$n = 50$ → 9 PB（単精度複素数）
    - 計算量: $O(m \exp n)$　$n = 50, m = 1000$ → 1 EFLOP



- テンソル表現で疎行列を効率的に扱える

Feynman シミュレーション

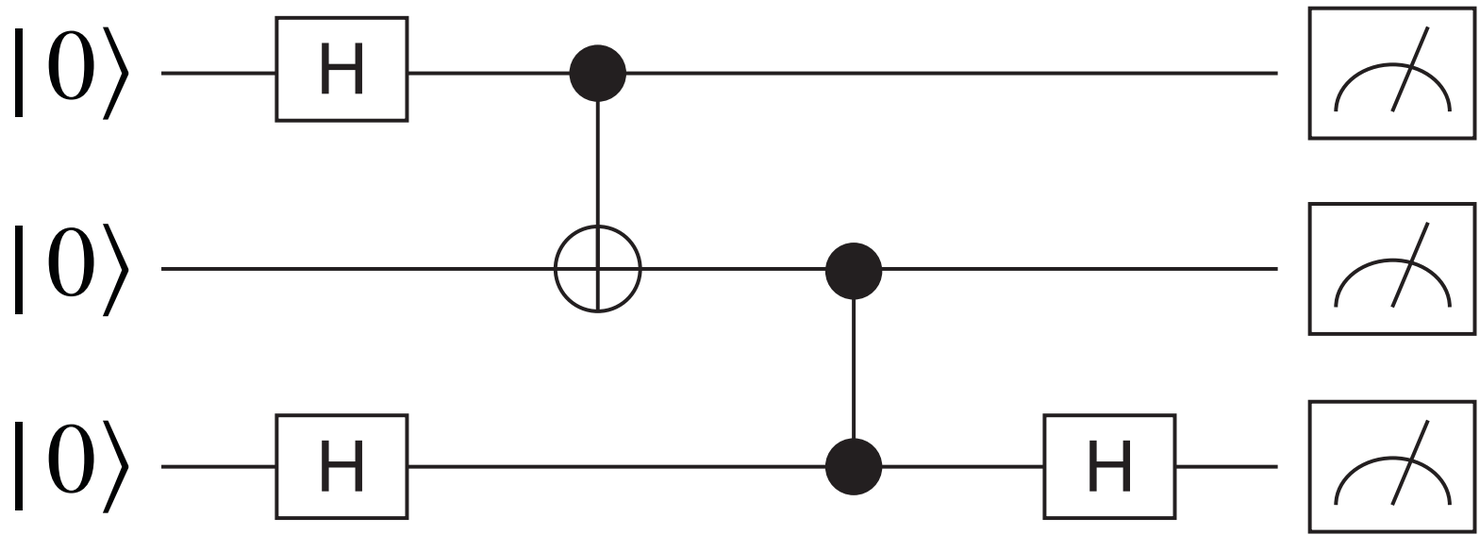
- すべての「経路」からの寄与を足し合わせる
  - 量子力学の経路積分に対応
  - Hadamard のような分岐ゲートを通るたびに、異なる経路へ分岐する
  - 各経路の寄与（重み）は行列要素の積で与えられる
- シミュレーションコスト
  - n qubits, m 個の分岐ゲート
    - 経路数: $O(\exp m)$
    - メモリコスト: $O(n)$ （何を測るかに依存）
    - 計算量: $O(n \exp m)$



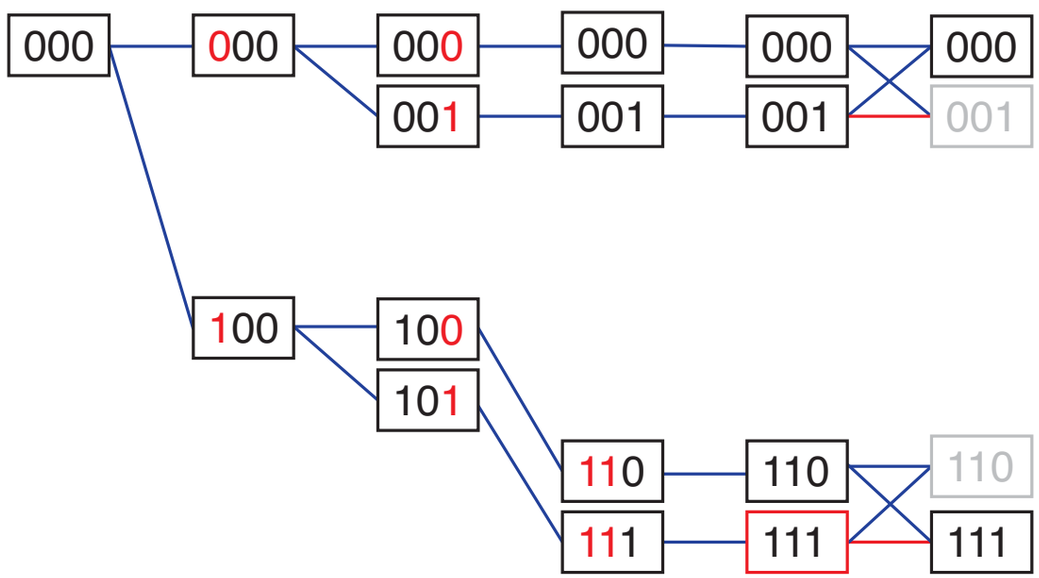
Feynman サンプリング

- 別名 Quantum Monte Carlo
  - 各分岐において, 行列要素に比例した確率で次状態を確率的に選ぶ
- シミュレーションコスト
  - n qubits, m 個の分岐ゲート
    - 1サンプルあたりの計算量: $O(m)$
    - 総計算量: $O(m)$ x (サンプル数)
- 符号問題
  - 行列要素が負(あるいは複素数)のときどうするか？
    - 行列要素の絶対値を「重み」として使う
    - 符号(または位相)は別に保持する
    - 符号が混ざると強い相殺が起きる
  - 回路が深くなると, ほとんどランダムサンプリングになってしまう
    - 必要サンプル数: $O(\exp m)$

量子回路の経路サンプリング



- 状態遷移図



- 干渉によって多くの状態が消える→ 負符号(複素位相)問題

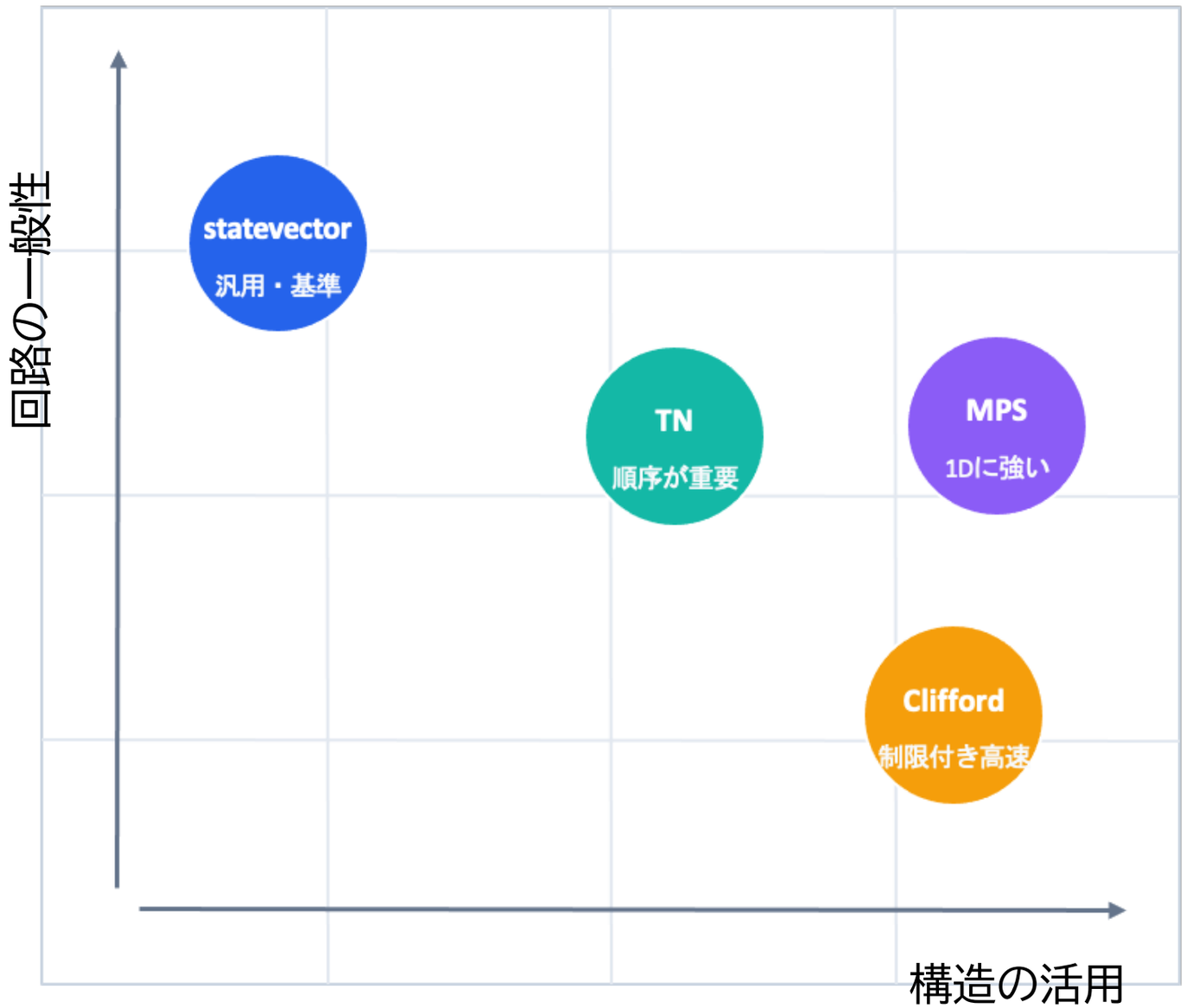
よりモダンな手法

- テンソルネットワークに基づく方法
  - MPS（TEBD）
    - 一次元的な回路・エンタングルメントが小さい場合に強い
    - 長距離相互作用がある場合、回路が深い場合は破綻する
  - （狭義の）テンソルネットワーク法
    - 量子回路全体をテンソルネットワークと見立て、縮約を行う
    - 縮約順序が性能を大きく左右する
    - 幅が狭い回路・浅い回路で有効
- スタビライザー表現に基づく方法
  - Cliffordシミュレータ
    - 非常に高速
    - 限られた種類の回路（=Clifford回路）しか扱えない

シミュレーション手法の地図

- 大規模回路では statevector だけではすぐ限界に達する
- 重要なのは「構造を使う」「必要な量だけ計算する」「厳密と近似を使い分ける」

- キーワード
  - 縮約順序
  - slicing
  - Clifford
  - 近似



TN シミュレータ

TNシミュレータ

- 量子回路全体を大きなテンソルネットワークと見立て、縮約を行う
  - 縮約順序が性能を大きく左右する
  - 悪い順序: 巨大な中間テンソルの発生
  - 良い順序: 木幅を小さく維持

- 例: 行列積における縮約順序の重要性
  - ABC = (AB)C = A(BC)？
  - A = 2 x 1000, B = 1000 x 2, C = 2 x 1000
  - (AB)Cの場合
    - AB: 2 x 1,000 x 2, (AB)C: 2 x 2 x 1,000, 合計 8,000
  - A(BC)の場合
    - 途中で 1000 x 1000 の巨大行列が出現
    - BC: 1,000 x 2 x 1,000, A(BC): 2 x 1,000 x 1,000, 合計 4,000,000
  - 最終結果は一致するにもかかわらず、順序の違いで計算コストに「500倍」の差が生じる

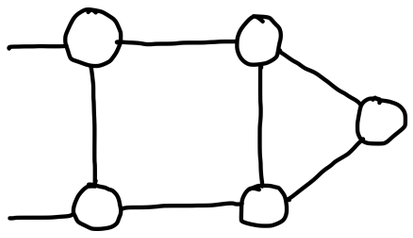
最適縮約順序の検索

- 候補数は膨大であり、総当たりによる完全な探索は非現実的
- 「最適解」より「十分よい順序を早く見つける」ことが重要
- 代表的なヒューリスティックな手法
  - 貪欲法（中間テンソルサイズ重視）
  - Random-greedy
  - 分割統治・グラフ分割
- ヒューリスティックな手法を実装した代表的なライブラリ
  - opt_einsum
  - cotengra
  - KaHyPar
  - NCON

テンソルネットワークの「スライシング」

- スライシング（Slicing）
  - テンソルネットワークの中のボンドの部分集合について
    - それらのボンド変数をある値に固定する
    - 固定されたボンドはテンソルネットワークから消える
  - 固定された値の全ての組み合わせについて
    - 独立に取り足し合わせる
- スライシングの利点
  - メモリコストを減らすことが可能
  - それぞれの組み合わせに関する縮約は（並列コンピュータ上で）並列に計算可能

- 例:



Clifford シミュレータ

Clifford演算子

- N量子ビット系に対して
  - Pauli 群 P_N

$$\mathcal{P}_N = \{\pm 1, \pm i\}\{I, X, Y, Z\}^{\otimes N}$$

  - Clifford 群 C_N
    - Pauli 群を正規化するユニタリ群(大域位相を除いて考える)

$$\mathcal{C}_N = \{U \in \mathrm{SU}(2^N) \,|\, UPU^\dagger \in \mathcal{P}_N \quad \forall \ P \in \mathcal{P}_N\}$$

  - C_N の生成元
    - Hadamard (H), $\pi/4$ 位相回転 (S), 制御位相ゲートCZ（ΛZ）
    - これらにより任意の Clifford 演算子を生成できる

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} \quad \Lambda Z = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$$

    - 例) $HXH^\dagger = Z, HZH^\dagger = X, SXS^\dagger = Y, SZS^\dagger = Z$
- N=1 (2) では、Clifford演算子は24 (11,520)個

Gottesman-Knillの定理

- Gottesman (1998)
  - 次の要素だけからなる量子回路(stabilizer回路)は、古典計算機で効率的にシミュレートできる
    - 計算基底状態での量子ビット準備
    - Clifford 群に属する量子ゲート
    - 計算基底での測定

Stabilizer状態

Hands-on: 41_clifford

- Stabilizer状態
  - 互いに可換で独立な N 個のエルミートな(すなわち位相が ± 1 の) N ビット Pauli 演算子(固有値 ± 1)の $+1$ 同時固有状態

$$P_a|\psi\rangle = |\psi\rangle, \quad P_a \in \mathcal{P}_N, \quad a = 1, \ldots, N \qquad\qquad P_a^\dagger = P_a\,, \;\; P_a^\dagger = P_a^{-1}$$

  - 3-qubit の例
    - $|000\rangle : P_1 = ZII, P_2 = IZI, P_3 = IIZ$, $|111\rangle : P_1 = -ZII, P_2 = -IZI, P_3 = -IIZ$
    - $(|000\rangle + |111\rangle)/\sqrt{2} : P_1 = XXX, P_2 = ZZI, P_3 = IZZ$
- Cliffordゲートの作用
  - 計算基底状態は stabilizer 状態である
  - Clifford ゲート U が stabilizer 状態に作用すると, 結果も stabilizer 状態である
    - 新しいstabilizerは、$UP_aU^\dagger \in \mathscr{P}_N$
    - $[P_a, P_b] = 0 \Rightarrow [UP_aU^\dagger, UP_bU^\dagger] = U[P_a, P_b]U^\dagger = 0$
    - $(UP_aU^\dagger)U|\Psi\rangle = UP_a|\Psi\rangle = U|\Psi\rangle$

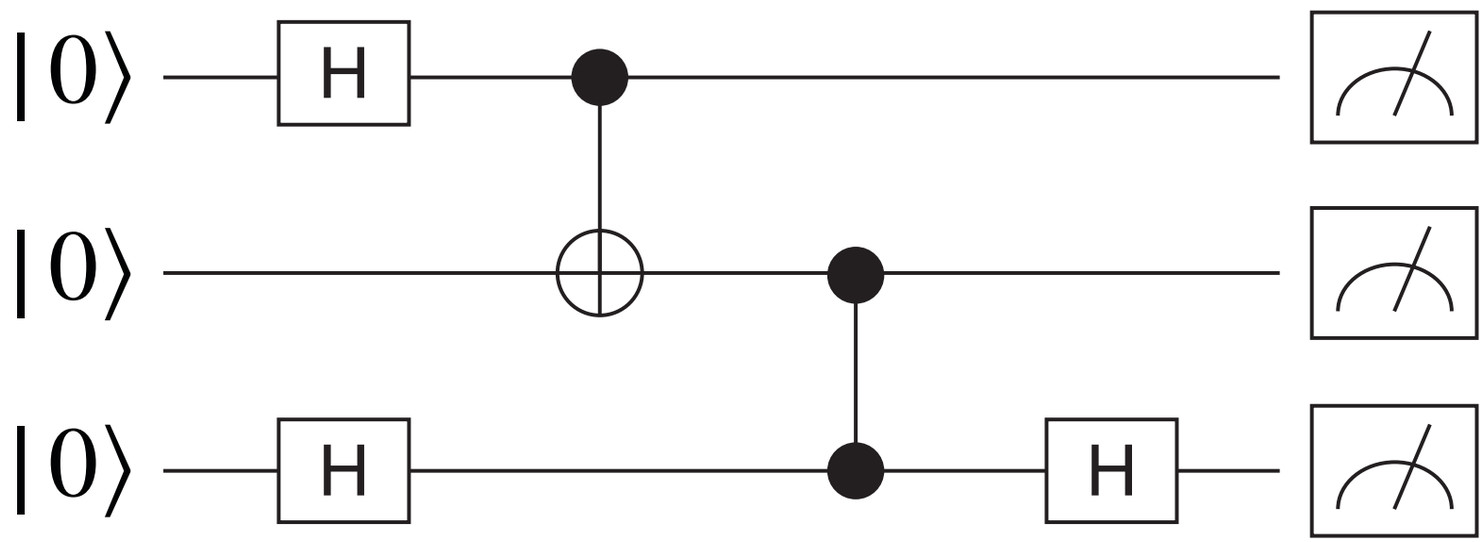
Stabilizer tableau シミュレータ

- Stabilizer tableau
  - stabilizer回路の状態は常にstabilizer tableauで表せる
  - $\{I, X, Y, Z\}$ からなる $N \times N$ 演算子テーブルと符号因子 ± 1
- Stabilizer tableau シミュレータ
  - 状態(=2^N個の係数)を更新する代わりに, stabilizer tableau (=$N \times N$行列)を更新する
    - メモリコスト = $N^2 \ll 2^N$
    - CPU コスト = $N \times$ #operators

	1	2	3	4
$+$	Z	Z	X	I
$+$	X	X	X	I
$-$	X	Z	Y	Z
$+$	I	I	X	Y

Aaronson, Gottesman (2004)

例: statevector



- $|000\rangle \Rightarrow \frac{1}{2}(|0\rangle + |1\rangle)|0\rangle(|0\rangle + |1\rangle) = \frac{1}{2}(|000\rangle + |001\rangle + |100\rangle + |101\rangle)$

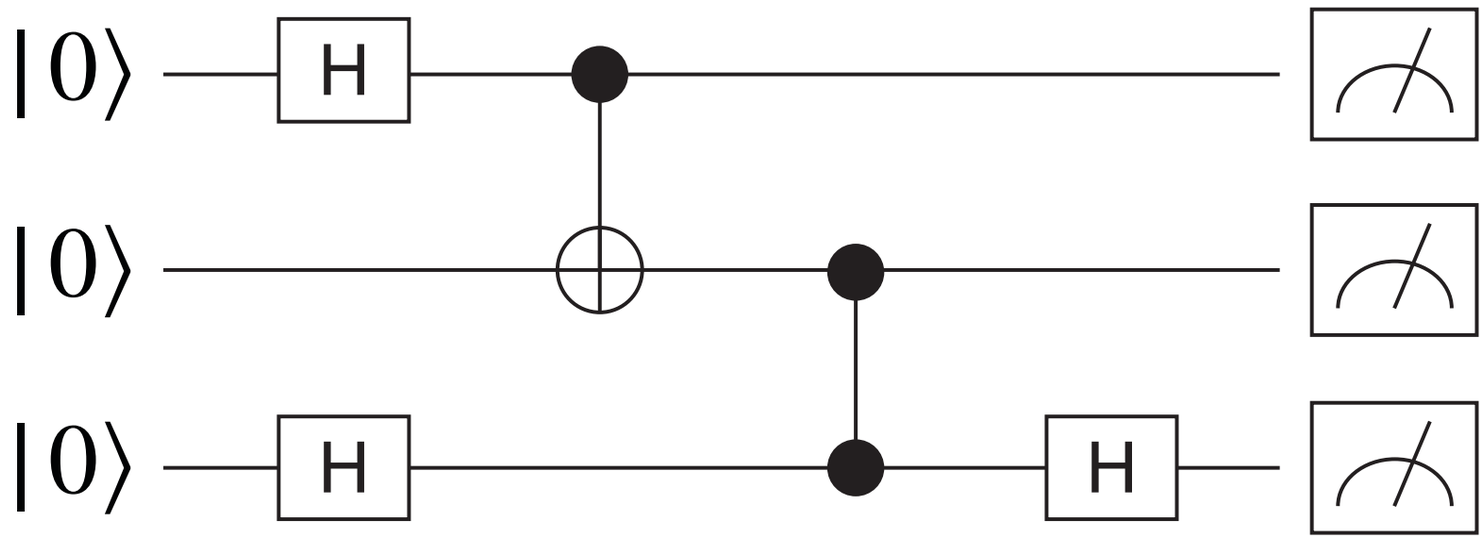
$$\Rightarrow \frac{1}{2}(|000\rangle + |001\rangle + |110\rangle + |111\rangle) \;\; \Rightarrow \frac{1}{2}(|000\rangle + |001\rangle + |110\rangle - |111\rangle)$$

$$\Rightarrow \frac{1}{2\sqrt{2}}\left[(|00\rangle + |11\rangle)(|0\rangle + |1\rangle) + (|00\rangle - |11\rangle)(|0\rangle - |1\rangle)\right]$$

$$= \frac{1}{2\sqrt{2}}(|000\rangle + |001\rangle + |110\rangle + |111\rangle + |000\rangle - |001\rangle - |110\rangle + |111\rangle)$$

$$= \frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$$

例: stabilizer tableau



+	Z	I	I
+	I	Z	I
+	I	I	Z

$\xrightarrow{H_1}$

+	X	I	I
+	I	Z	I
+	I	I	Z

$\xrightarrow{H_3}$

+	X	I	I
+	I	Z	I
+	I	I	X

$\xrightarrow{CX_{12}}$

+	X	X	I
+	Z	Z	I
+	I	I	X

$\xrightarrow{CZ_{23}}$

+	X	X	Z
+	Z	Z	I
+	I	Z	X

$\xrightarrow{H_3}$

+	X	X	X
+	Z	Z	I
+	I	Z	Z

$$= \frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$$

より効率的な表現: グラフ状態

- N量子ビットのグラフ状態 $|G\rangle$
  - 数学的なグラフ$G=(V,E)$ ($|V|=N$)から定義される量子状態
  - N個の固有値方程式の一意解として定義される

$$K_G^{(a)}|G\rangle = |G\rangle, \quad a \in V$$

$$\text{with } K_G^{(a)} = \sigma_x^{(a)} \prod_{b \in \text{ngbh } a} \sigma_z^{(b)} =: X_a \prod_{b \in \text{ngbh } a} Z_b$$

- グラフ状態の生成

$$|G\rangle = \left(\prod_{\{a,b\} \in E} \Lambda Z_{ab} \right) \left(\prod_{a \in V} H_a \right) |0\rangle^{\otimes N}$$

- グラフ状態表現
  - すべてのグラフ状態は stabilizer 状態である
  - さらに任意の stabilizer 状態は、グラフ状態 + 局所 Clifford 演算子として記述できる

$$|G;\underline{C}\rangle := |G;C_1,C_2,\ldots,C_N\rangle := \bigotimes_{i=1}^{N} C_i|G\rangle$$

Hein, Eisert, Briegel (2004)

グラフ状態シミュレータ

- stabilizer状態
  - 常に「グラフ状態 + 局所 Clifford 演算子」として表せる
- グラフ状態シミュレータ
  - 初期状態

$$|0\rangle^{\otimes N} = |(\{1,\ldots,N\},\{\});H,\ldots,H\rangle$$

  - stabilizer tableau を更新する代わりに, グラフと局所 Clifford 演算子を更新する
    - メモリコスト $= N \times \bar{d}$ （$\bar{d}$: 頂点の平均次数, $\ll N$）
    - 計算量 $= N \times \bar{d} \times$ #operations
  - グラフ状態表現で扱える量子ビット数が大きく伸びる

Anders, Briegel (2006)

Clifford+T シミュレータ

- Clifford ゲートだけでは普遍的な量子ゲート集合にならない
  - 例: 位相シフトゲート（$\pi/8$ゲート）T は Clifford 演算子ではなく、Clifford演算子では近似できない

$$T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\frac{\pi}{4}} \end{bmatrix} = \sqrt{S} = \sqrt[4]{Z}.$$

  - しかし、Clifford群 + Tゲートは、ユニバーサル量子ゲート集合を成す
- Clifford+T シミュレータ
  - 各Tゲートを Clifford **チャネル**の線形結合として扱う

$$T = I\cos\theta + S\frac{1-\sin\theta-\cos\theta}{2} + S^\dagger\frac{1+\sin\theta-\cos\theta}{2} \qquad \text{with } \theta = \pi/8$$

  - Clifford ゲートの組合せについて平均を取る
  - 計算量 = $N \times d \times$ #operations $\times \exp(\#\text{T-gates})$